하이브리드 AI**기반 악성코드 판별**

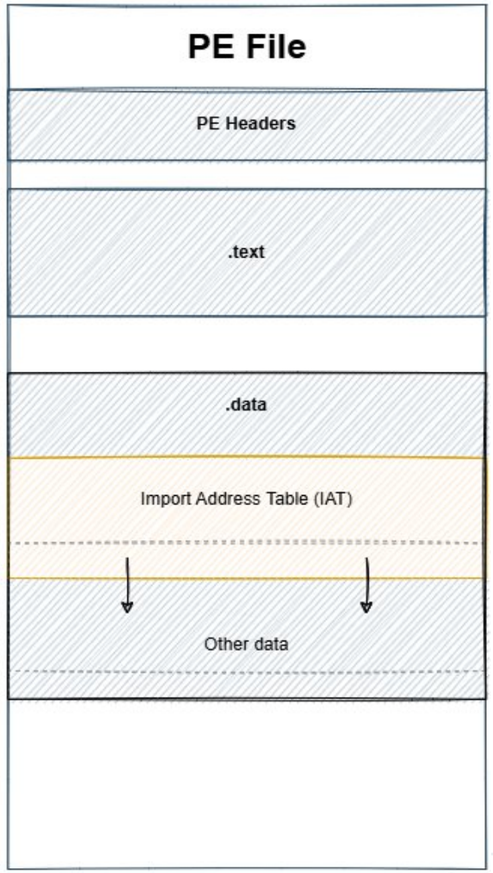
- API 호출 시퀀스 분석

Cybersecurity at
Your Fingertips

Understanding and
Preventing Cyber
Attacks

왜 API 시퀀스인가?

- IAT에서 얻은 영감



개별 API의 존재 유무가 아니라 실제 실행 환경에서는 프로그램의 내부 함수들이 특정 로직에 따라 순서대로 호출될 것이며, 각 함수는 IAT에 명시된 API들을 필요한 시점에 호출할 것이라고 추론했습니다. 예를 들어, InternetOpen -> URLDownloadToFile -> WinExec으로 이어지는 API 호출의 순서는, 이 API들이 단순히 IAT 목록에 섞여있는 것보다 훨씬 더 강력하고 구체적인 행위의 증거가 됩니다. 이 아이디어에 착안하여, 단순히 IAT에 어떤 API가 있는지를 보는 정적인 목록 분석을 넘어, 함수와 함수 간의 호출 관계를 추적하여 이 API들이 어떤 순서로 호출될 것인지를 예측하는 API 시퀀스를 핵심 특징으로 삼게 되었습니다. 이것이 바로 최초의 출발점이자 가장 중요한 이론적 배경입니다.

정적분석의 한계

- 패킹과 난독화

구분	정적 분석의 한계 및 단점	동적 분석을 사용하는 목적
분석 시점	코드 실행 없이 분석하므로 런타임 환경에서 발생하는 오류를 탐지할 수 없음	소프트웨어를 직접 실행하여 런타임 중에 발생하는 오류를 탐지
분석 대상	코드의 구조, 문법, 코딩 표준 위반 등 정적인 측면에 집중	실제 데이터 입력 및 사용자 상호작용에 따른 프로그램의 동적인 동작 및 행위를 분석
탐지 가능 오류	복잡한 로직, 외부 라이브러리 연동, 설정 값 등에 의해 발생하는 복합적인 취약점 탐지에 한계가 있음	실제 공격 시나리오를 적용하여 논리적 허점이나 인증/인가 문제와 같은 복잡한 보안 취약점을 발견
정확성	실제 실행 환경을 고려하지 못해 탐지된 오류가 실제 위협이 아닌 긍정 오류일 가능성이 있음	실제 환경에서 직접 테스트하므로 탐지된 오류가 실제 문제일 가능성이 높고, 프로그램의 기능적 동작을 정확히 검증 가능
코드 난독화	코드가 난독화되어 있을 경우 분석의 정확도가 현저히 떨어지거나 분석이 어려움	코드의 내부 구조와 상관없이 프로그램의 입출력 및 행위에 기반하여 분석하므로 난독화의 영향을 덜 받음

이상적 접근법

- CAPEv2 **샌드박스: 행위 기반 동적 분석**

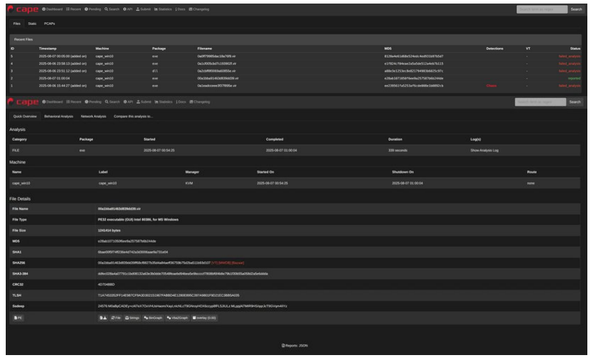
CAPEv2 Sandbox

CAPE is an open-source automated malware analysis system. It's used to automatically run and analyze files and collect comprehensive analysis results that outline what the malware does while running inside an isolated Windows operating system.

이상적인 해결책으로, CAPEv2와 같은 자동화된 동적분석 샌드박스 도입을 초기 목표로 설정했습니다. 샌드박스는 격리된 가상 환경에서 파일을 직접 실행하여 API 호출, 파일 시스템 I/O, 네트워크 트래픽 등 구체적인 행위의 흔적을 실시간으로 수집합니다. 이렇게 확보된 실제 API 호출 시퀀스는 악성 행위를 규정하는 가장 신뢰도 높은 특징이며 이를 통해 고성능 행위 기반 탐지 모델을 구축하고자 했습니다.

현실적 대안

- 정적 분석을 통한 행위 예측



그러나 샌드박스 운영에 필요한 높은 컴퓨팅 자원과 긴 분석 시간(10분 이상 소요)은 프로젝트의 목적에 적합하지 않았습니다.

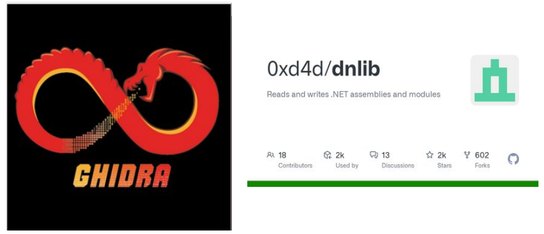
Native 파일 분석 (Ghidra): NSA의 리버스 엔지니어링 프레임워크인 Ghidra의 디컴파일러를 활용하여 바이너리의 함수 호출 그래프(Call Graph)를 재구성했습니다. 이후 DFS(깊이 우선 탐색) 알고리즘을 적용하여 가장 확률 높은 실행 경로를 추론함으로써, 실제 실행 없이도 동적 분석과 유사한 예측된 API 호출 시퀀스를 생성했습니다.

NET 파일 분석 (dnlib): .NET 어셈블리는 메타데이터가 풍부하다는 점에 착안, 전문 라이브러리인 dnlib을 통해 CIL(중간 언어)을 직접 파싱했습니다. 이를 통해 Native 분석보다 훨씬 빠르고 정확하게 API 및 P/Invoke 호출 정보를 추출하여 동일한 구조의 시퀀스로 표준화했습니다.

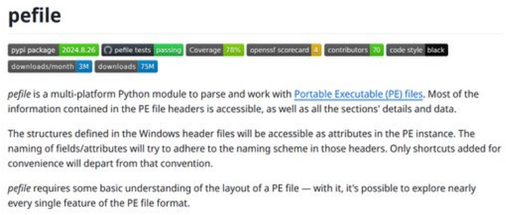
api_sequence_model

한계를 넘어

- 하이브리드 AI 모델을 선택한 이유



<정적 행위 모델>



<정적 구조 모델>

단일 분석 방식이 가질 수 있는 탐지 공백을 최소화하기 위해, 서로 다른 관점에서 파일을 분석하는 두 개의 독립적인 AI 모델을 결합한 하이브리드 접근법을 선택하였습니다.

1. API 시퀀스 분석 모델 (행위 기반 예측)

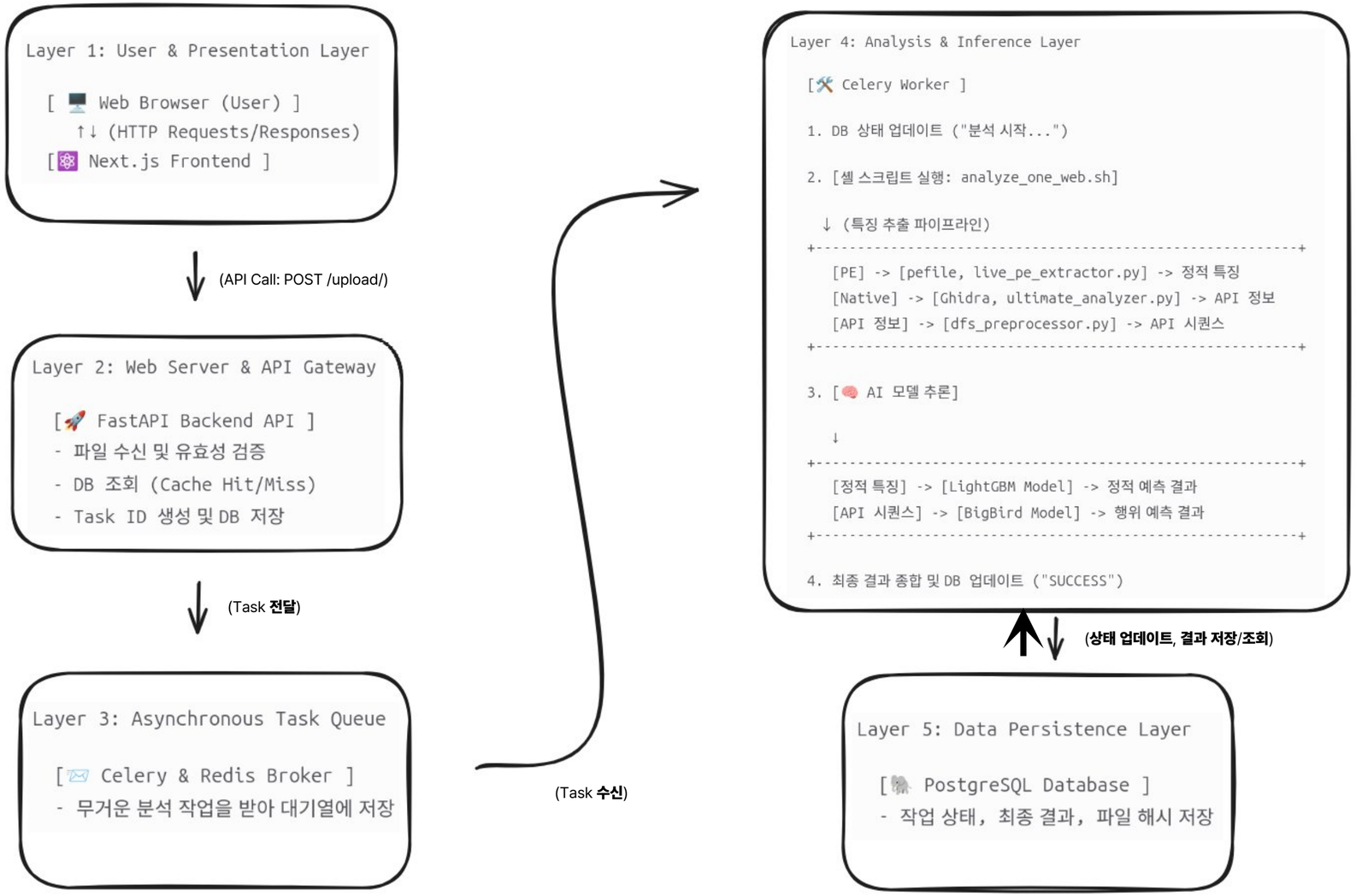
- 역할: 이 모델은 Ghidra 및 dnlib으로 추출한 함수 호출 관계를 기반으로, 프로그램의 예측된 실행 흐름을 분석합니다. API 호출들을 순차적인 데이터로 간주하고, 특정 API 호출들의 조합과 순서가 악성 행위 패턴과 얼마나 일치하는지를 학습합니다.
- 강점: 정상적인 API들을 특정 순서로 조합하여 악의적인 기능을 수행하는 논리적 공격을 탐지하는 데 효과적입니다. 예를 들어, 파일 다운로드, 실행, 지속성 유지로 이어지는 일련의 행위 패턴을 식별할 수 있습니다.
- 보완점: 이 방식은 코드의 실행 경로를 예측하는 데 의존하므로, 정적 분석만으로 파악하기 어려운 복잡한 분기 조건 하에 숨겨진 악성 코드를 탐지하지 못할 수 있습니다. 또한, 실행 흐름에 포함되지 않은 데이터 영역의 잠재적 위협은 분석 대상에서 제외됩니다.

2. PE 구조 분석 모델 (정적 특징 분석)

- 역할: 이 모델은 파일이 실행되기 전 상태의 구조적, 통계적 무결성을 검사합니다. PE 헤더의 값, 섹션의 개수와 속성, 데이터의 엔트로피, 디지털 서명의 유효성 등 파일의 물리적인 속성을 특징으로 사용합니다.
- 강점: 실행 압축이나 코드 암호화가 적용된 파일을 탐지하는 데 강점을 보입니다. 이러한 파일들은 비정상적으로 높은 엔트로피 값을 갖거나, 일반적인 PE 구조와 다른 특징을 나타내기 때문입니다. 파일 실행 없이도 잠재적으로 위험한 구조를 가진 파일을 빠르게 식별할 수 있습니다.
- 보완점: 파일의 구조가 완벽하게 정상적이더라도, 논리적인 흐름 자체가 악의적인 목적으로 설계된 경우는 탐지하기 어렵습니다.

이 두 모델을 결합함으로써, 저희 시스템은 구조적으로 의심스럽지만 행위 예측은 어려운 경우와, 구조는 정상이지만 행위 패턴이 악의적인 경우를 모두 포괄할 수 있습니다. 이처럼 상호 보완적인 분석을 통해 단일 모델 접근법보다 더 넓은 탐지 범위를 확보하고, 오탐을 줄여 전체 시스템의 강건함과 신뢰도를 극대화했습니다.

시스템 아키텍처



기술 스택 선정 이유

- **FastAPI & Celery: 시간이 오래 걸리는 분석 작업 중에도 사용자가 웹사이트를 쾌적하게 이용할 수 있도록 비동기 처리 아키텍처를 채택**

- **Dual AI Models : 각 특징의 형태에 최적화된 모델을 사용. (정형 데이터 -> Tree 기반 모델, 순차 데이터 -> Transformer 기반 모델)**

데이터 수집

정상파일

server_22.zip	2.3 GB
server_25.zip	3.3 GB
win_10.zip	2.9 GB
win_11.7z	2.0 GB

benign	12,428 items
malware	49,787 items
benign.7z	2.4 GB
malware.7z	12.8 GB

악성파일

2024-01-01.zip	2024-01-02 00:02 174M
2024-01-02.zip	2024-01-03 00:03 234M
2024-01-03.zip	2024-01-04 00:04 224M
2024-01-04.zip	2024-01-05 00:03 235M
2024-01-05.zip	2024-01-06 00:06 434M
2024-01-06.zip	2024-01-07 00:03 281M
2024-01-07.zip	2024-01-08 00:02 194M
2024-01-08.zip	2024-01-09 00:04 368M
2024-01-09.zip	2024-01-10 00:05 435M

출처:https://datalake.abuse.ch/malware-bazaar/daily/

구분	수집처	갯수
benign	**win_10, win_11, server_22, server_25 dll, exe 파일 수집**	**12,428**
malware	**bazzar/daily 2024-2025년도 dll, exe 파일 수집**	**49,787**

구조 모델 - Features

PE 파일 특징 상세 설명표

특징 (Feature)	설명
Machine	파일이 실행되도록 컴파일된 CPU 아키텍처 (예: x86, x64)
NumberOfSections	파일 내부에 존재하는 코드 및 데이터 영역(섹션)의 총 개수
TimeDateStamp	파일이 컴파일된 시간 정보 (1970년 1월 1일 이후 경과 시간)
Characteristics	파일의 속성을 나타내는 플래그 (예: 실행 파일, DLL 파일 여부)
SizeOfCode	실행 코드를 포함하는 모든 섹션의 전체 크기
SizeOfInitializedData	초기화된 데이터를 포함하는 모든 섹션의 전체 크기
SizeOfUninitializedData	초기화되지 않은 데이터를 포함하는 모든 섹션의 전체 크기
AddressOfEntryPoint	프로그램 실행이 시작되는 메모리 주소
ImageBase	파일이 메모리에 로드될 때 권장되는 시작 주소
SizeOfImage	파일이 메모리에 로드되었을 때 차지하는 전체 메모리 크기
SizeOfHeaders	PE 헤더(파일 구조 정보) 영역의 전체 크기
Subsystem	파일 실행에 필요한 하위 시스템 (예: GUI-그래픽, CUI-콘솔)
DllCharacteristics	ASLR, DEP 등 보안 기능 적용 여부를 나타내는 DLL 속성 플래그
SizeOfStackReserve	프로세스를 위해 예약된 스택 메모리의 최대 크기
SizeOfHeapReserve	프로세스를 위해 예약된 힙 메모리의 최대 크기
num_sections	섹션의 총 개수 (NumberOfSections와 동일)
section_entropy_mean	모든 섹션의 엔트로피(데이터의 무질서도 또는 압축률) 평균값
section_entropy_std	모든 섹션 엔트로피의 표준편차
section_size_mean	모든 섹션 크기의 평균값
section_size_std	모든 섹션 크기의 표준편차
num_executable_sections	실행(Execute) 권한을 가진 섹션의 개수
executable_entropy_mean	실행 권한을 가진 섹션들의 평균 엔트로피
num_wx_sections	쓰기(Write)와 실행(Execute) 권한을 동시에 가진 섹션의 개수
num_imports	외부 라이브러리(DLL)로부터 가져오는 함수의 총 개수
num_imported_libs	함수를 가져오는 대상이 되는 외부 라이브러리(DLL)의 총 개수
num_exports	다른 프로그램이 사용할 수 있도록 외부에 제공하는 함수의 총 개수
is_signed	파일에 디지털 서명이 있는지 여부 (1: 있음, 0: 없음)
is_self_signed	서명이 공인 인증 기관이 아닌 자체 발급되었는지 여부 (1: 맞음, 0: 아님)
signature_valid	디지털 서명의 구문 분석 성공 여부로 판단한 간이 유효성
cert_validity_days	서명에 사용된 인증서의 유효 기간 (일수)
issuer_is_suspicious	인증서 발급자 이름에 'test' 등 의심스러운 키워드 포함 여부
subject_is_suspicious	인증서 주체(소유자) 이름에 의심스러운 키워드 포함 여부

```
전문가 수준 특징 추출 및 균형 데이터 분할 시작
================================================================================

[1/4] '/shared/data/'에서 클래스별 파일 목록 수집...
  → 악성코드: 49787개, 정상: 12428개 파일 발견.

[2/4] 학습용/검증용 데이터셋 균형 샘플링...
  → 학습용 데이터: 20000개 (악성 10000, 정상 10000)
  → 홀드아웃 데이터: 2000개 (악성 1000, 정상 1000)

[3/4] 학습용 데이터셋 특징 추출...

  (학습 데이터 분석) 7개 프로세스로 분석 시작...
학습 데이터 분석: 100%|██████████████████████████████| 20000/20000 [04:34<00:00, 72.93it/s]

  → (학습 데이터 분석) 완료! 19980개 파일 특징을 '/home/jy/csv/pro_train_features.csv'에 저장.

[4/4] 홀드아웃 데이터셋 특징 추출...

  (홀드아웃 데이터 분석) 7개 프로세스로 분석 시작...
홀드아웃 데이터 분석: 100%|██████████████████████████████| 2000/2000 [00:26<00:00, 74.15it/s]

  → (홀드아웃 데이터 분석) 완료! 1995개 파일 특징을 '/home/jy/csv/pro_holdout_features.csv'에 저장.
```

정상, 비정상 파일 각 10000개씩 피처 추출

구조 모델 - Modeling

```
[1/4] 데이터 로딩 및 전처리...

[정보] 학습 데이터에서 항상 0인 피처들을 제거합니다: ['num_imports', 'num_imported_libs', 'num_exports', 'is_self_signed', 'signature_valid', 'issuer_is_suspicious', 'subject_is_suspicious']
  → 데이터 로딩 및 전처리 완료.

[2/4] 모델 튜닝 및 최고 모델 선정 (확장된 탐색)...

--- LightGBM 튜닝 중 ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits
[LightGBM] [Info] Number of positive: 7980, number of negative: 8000
[LightGBM] [Info] This is the GPU trainer!!
[LightGBM] [Info] Total Bins 3248
[LightGBM] [Info] Number of data points in the train set: 15980, number of used features: 24
[LightGBM] [Info] Using GPU Device: NVIDIA A100-SXM4-80GB, Vendor: NVIDIA Corporation
[LightGBM] [Info] Compiling OpenCL Kernel with 256 bins...
[LightGBM] [Info] GPU programs have been built
[LightGBM] [Info] Size of histogram bin entry: 8
[LightGBM] [Info] 19 dense feature groups (0.30 MB) transferred to GPU in 0.001320 secs. 1 sparse feature groups
[LightGBM] [Info] [binary:BoostFromScore]: pavg=0.499374 -> initscore=-0.002503
[LightGBM] [Info] Start training from score -0.002503

  → LightGBM 최적 파라미터: {'reg_lambda': 0, 'reg_alpha': 0, 'num_leaves': 31, 'n_estimators': 500, 'learning_rate': 0.05}
  → LightGBM 테스트 F1 Score: 0.9907
  🎉 *** 새로운 최고 성능 모델: LightGBM ***

--- RandomForest 튜닝 중 ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits

  → RandomForest 최적 파라미터: {'n_estimators': 100, 'min_samples_split': 2, 'min_samples_leaf': 2, 'max_depth': 20}
  → RandomForest 테스트 F1 Score: 0.9882

--- CatBoost 튜닝 중 ---
Fitting 3 folds for each of 10 candidates, totalling 30 fits

  → CatBoost 최적 파라미터: {'learning_rate': 0.1, 'l2_leaf_reg': 5, 'iterations': 1000, 'depth': 8}
  → CatBoost 테스트 F1 Score: 0.9895
```

32개의 피처 중 피처값이 전부 0인 7개의 피처를 제외한 25개의 피처로 학습한 결과 F1 Score값이 제일 높은 LightGBM모델 선정

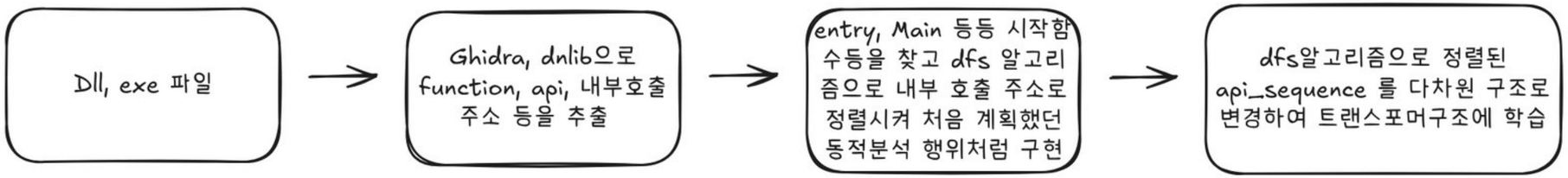
❖ F1 score 지표 선정 이유: 데이터가 불균형하기 때문

```
[3/4] 홀드아웃 데이터로 최종 성능 검증...

================================================================================
          모델 일반화 성능 최종 검증: 홀드아웃(Hold-out) 데이터셋
================================================================================

  • 최고 모델: LightGBM
  • 내부 테스트셋 F1 Score: 0.9907
  • ★★★ 최종 홀드아웃셋 F1 Score: 0.9925 ★★★

  • 최종 홀드아웃 분류 리포트:
              precision    recall  f1-score   support

      Benign       0.99      0.99      0.99      1000
     Malware       0.99      0.99      0.99       997

    accuracy                           0.99      1997
   macro avg       0.99      0.99      0.99      1997
weighted avg       0.99      0.99      0.99      1997


  • 최종 홀드아웃 혼동 행렬:
[[992   8]
 [  7 990]]
================================================================================
```

스코어가 높아 과적합 의심되어 대략 12000개의 파일 중 학습과 검증에 사용된 10000개를 제외한 1997개의 파일을 예측한 결과 0.99 결과값 산출

행위 모델 - Features

- Processing



행위 모델 - Features

- Ghidra, Dnlib **추출 구조**


```
file_name:                "0a5250c69150753cfcf43475d72f4e1c7ae23ae4d7a71242cfc96fd693e3206c.exe"
md5:                      "d8add8b8a36363131385482a2074dad2"
analysis_status:          "success"
error:                    ""
file_type:                "native"
target:                   "malware"
functions:                (324)[ {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, … ]
analysis_time_seconds:    6.96
functions:
  0:
    name:                 "FUN_00404050"
    address:              "0x00404050"
    api_call_sequence:    [ "KERNEL32.DLL::LoadLibraryW" ]
    internal_call_sequence: []
  1:
    name:                 "FUN_004040f0"
    address:              "0x004040f0"
    api_call_sequence:    []
    internal_call_sequence: []
  2:
    name:                 "FUN_00404100"
    address:              "0x00404100"
    api_call_sequence:    (12)[ "KERNEL32.DLL::SetFileAttributesA", "KERNEL32.DLL::GetCommConfig",
                          "KERNEL32.DLL::GetCurrencyFormatW",
                          "KERNEL32.DLL::GetLogicalDriveStringsA",
                          "KERNEL32.DLL::GetComputerNameA", "KERNEL32.DLL::ClearCommBreak",
                          "KERNEL32.DLL::EnumTimeFormatsW", "KERNEL32.DLL::GetTempFileNameW",
                          "KERNEL32.DLL::WriteConsoleInputW", "KERNEL32.DLL::GetVersionExA", … ]
    internal_call_sequence:
      0:                  { name: "__alloca_probe", address: "0x00411940" }
      1:                  { name: "FUN_004040f0", address: "0x004040f0" }
  3:
    name:                 "FUN_00404300"
    address:              "0x00404300"
    api_call_sequence:    (11)[ "KERNEL32.DLL::GetNumberFormatA", "KERNEL32.DLL::OpenJobObjectA",
                          "KERNEL32.DLL::GetConsoleAliasExesA", "KERNEL32.DLL::CallNamedPipeA",
                          "KERNEL32.DLL::SetFileShortNameW", "KERNEL32.DLL::CreateProcessA",
                          "KERNEL32.DLL::GetTimeFormatA", "KERNEL32.DLL::GetModuleFileNameW",
                          "KERNEL32.DLL::TlsSetValue", "KERNEL32.DLL::SetEnvironmentVariableA", …
                          ]
    internal_call_sequence:
      0:                  { name: "__alloca_probe", address: "0x00411940" }
      1:                  { name: "FUN_00404100", address: "0x00404100" }
```


Feature	설명	역할 및 설계 의도
functions	프로그램을 구성하는 모든 함수의 분석 정보를 담는 최상위 배열	복잡한 프로그램을 실행 가능한 최소 단위인 함수로 분해하여 분석하기 위함입니다. 이는 전체 프로그램을 그래프(Graph)의 노드(Node) 집합으로 모델링하기 위한 첫 단계입니다.
name & address	각 함수의 고유한 이름과 메모리 주소	고유 식별 address는 함수 간의 호출 관계를 기계적으로 매핑하는 고유 키(Key) 역할을 합니다. name은 분석 및 디버깅 과정에서 사람이 식별하기 위한 레이블(Label) 역할을 수행합니다.
api_call_sequence	해당 함수가 직접 호출하는 외부 시스템 함수의 목록	이 목록은 각 함수가 수행하는 가장 구체적이고 원자적인 '행동'을 명시합니다. AI 모델이 악성 행위를 판단하는 가장 직접적이고 핵심적인 특징 입니다.
internal_call_sequence	해당 함수가 호출하는 다른 내부 함수들의 참조 목록	함수와 함수를 잇는 연결고리입니다. 저희는 이 internal_call_sequence 정보를 따라 깊이 우선 탐색(DFS)을 수행함으로써, 단편적인 API 호출들을 하나의 의미 있는 예측 실행 경로, 즉 API 시퀀스로 재구성합니다. 이 구조가 없으면 API 호출의 순서와 문맥을 파악할 수 없습니다.

행위 모델 - Features

- DFS**알고리즘으로 정렬**


```
functions:
  0:
    name:                    "FUN_00404050"
    address:                 "0x00404050"
    api_call_sequence:       [ "KERNEL32.DLL::LoadLibraryW" ]
    internal_call_sequence:  []
  1:
    name:                    "FUN_004040f0"
    address:                 "0x004040f0"
    api_call_sequence:       []
    internal_call_sequence:  []
  2:
    name:                    "FUN_00404100"
    address:                 "0x00404100"
    api_call_sequence:       (12)[ "KERNEL32.DLL::SetFileAttributesA", "KERNEL32.DLL::GetCommConfig",
                             "KERNEL32.DLL::GetCurrencyFormatW",
                             "KERNEL32.DLL::GetLogicalDriveStringsA",
                             "KERNEL32.DLL::GetComputerNameA", "KERNEL32.DLL::ClearCommBreak",
                             "KERNEL32.DLL::EnumTimeFormatsW", "KERNEL32.DLL::GetTempFileNameW",
                             "KERNEL32.DLL::WriteConsoleInputW", "KERNEL32.DLL::GetVersionExA", … ]
    internal_call_sequence:
      0:                     { name: "__alloca_probe", address: "0x00411940" }
      1:                     { name: "FUN_004040f0", address: "0x004040f0" }
  3:
    name:                    "FUN_00404300"
    address:                 "0x00404300"
    api_call_sequence:       (11)[ "KERNEL32.DLL::GetNumberFormatA", "KERNEL32.DLL::OpenJobObjectA",
                             "KERNEL32.DLL::GetConsoleAliasExesA", "KERNEL32.DLL::CallNamedPipeA",
                             "KERNEL32.DLL::SetFileShortNameW", "KERNEL32.DLL::CreateProcessA",
                             "KERNEL32.DLL::GetTimeFormatA", "KERNEL32.DLL::GetModuleFileNameW",
                             "KERNEL32.DLL::TlsSetValue", "KERNEL32.DLL::SetEnvironmentVariableA", …
                             ]
    internal_call_sequence:
      0:                     { name: "__alloca_probe", address: "0x00411940" }
      1:                     { name: "FUN_00404100", address: "0x00404100" }
```


→


```
execution_trace:
  0:
    address:                 "0x404e53"
    original_address:        "0x00404e53"
    name:                    "___tmainCRTStartup"
    type:                    "call"
    depth:                   0
    apis:                    [ "KERNEL32.DLL::GetStartupInfoW" ]
  1:
    address:                 "0x405f1c"
    original_address:        "0x00405f1c"
    name:                    "__SEH_prolog4"
    type:                    "call"
    depth:                   1
    apis:                    []
  2:
    address:                 "0x407c2d"
    original_address:        "0x00407c2d"
    name:                    "__heap_init"
    type:                    "call"
    depth:                   1
    apis:                    [ "KERNEL32.DLL::HeapCreate" ]
  3:                         { address: "0x404e2a", original_address: "0x00404e2a", name: "fast_error_exit", … }
  4:                         { address: "0x406ed6", original_address: "0x00406ed6", name: "__FF_MSGBANNER", … }
  5:                         { address: "0x40c1c3", original_address: "0x0040c1c3", name: "__set_error_mode", … }
  6:                         { address: "0x40501c", original_address: "0x0040501c", name: "__errno", … }
  7:                         { address: "0x4078de", original_address: "0x004078de", name: "__getptd_noexit", … }
  8:                         { address: "0x407786", original_address: "0x00407786", name: "___set_flsgetvalue", … }
  9:                         { address: "0x40770b", original_address: "0x0040770b", name: "__decode_pointer", … }
  10:                        { address: "0x4069f3", original_address: "0x004069f3", name: "__crt_waiting_on_module_handle", … }
  11:                        { address: "0x40994a", original_address: "0x0040994a", name: "__calloc_crt", … }
```


internal_call_sequence를 깊이우선탐색(DFS) 수행 후
예측 실행 경로, 즉 API 시퀀스로 재구성

행위 모델 - Features

- 계층적 구조 변환

```
pre_df['execution_trace'][0]
```


```
'[{'address': '0x406a60', 'original_address': '0x00406a60', 'name': 'entry', 'type':
'call', 'depth': 0, 'apis': ['KERNEL32.DLL::GetStartupInfoW', 'KERNEL32.DLL::Sleep',
'MSVCRT.DLL::exit', 'MSVCRT.DLL::_ismbblead', 'MSVCRT.DLL::_cexit']}, {'address': '0x4
07155', 'original_address': '0x00407155', 'name': 'FUN_00407155', 'type': 'call', 'dep
th': 1, 'apis': ['KERNEL32.DLL::GetSystemTimeAsFileTime', 'KERNEL32.DLL::GetCurrentPro
cessId', 'KERNEL32.DLL::GetCurrentThreadId', 'KERNEL32.DLL::GetTickCount', 'KERNEL32.D
LL::QueryPerformanceCounter']}, {'address': '0x407208', 'original_address': '0x0040720
8', 'name': '__SEH_prolog4', 'type': 'call', 'depth': 1, 'apis': []}, {'address': '0x4
06ff4', 'original_address': '0x00406ff4', 'name': '_amsg_exit_THUNK', 'type': 'call',
'depth': 1, 'apis': ['MSVCRT.DLL::_amsg_exit']}, {'address': '0x406c3f', 'original_add
ress': '0x00406c3f', 'name': 'FUN_00406c3f', 'type': 'call', 'depth': 1, 'apis': []},
{'address': '0x4072a0', 'original_address': '0x004072a0',…'
```


→

```
df_processed['sequence_text'][0]
```

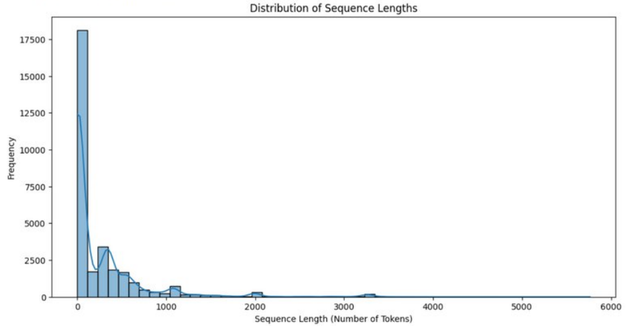

```
'FUNC_START::entry API::KERNEL32.DLL::GetStartupInfoW API::KERNEL32.DLL::Sleep API::MS
VCRT.DLL::exit API::MSVCRT.DLL::_ismbblead API::MSVCRT.DLL::_cexit FUNC_START::FUN_004
07155 API::KERNEL32.DLL::GetSystemTimeAsFileTime API::KERNEL32.DLL::GetCurrentProcessI
d API::KERNEL32.DLL::GetCurrentThreadId API::KERNEL32.DLL::GetTickCount API::KERNEL32.
DLL::QueryPerformanceCounter FUNC_END::FUN_00407155 FUNC_START::__SEH_prolog4 FUNC_EN
D::__SEH_prolog4 FUNC_START::_amsg_exit_THUNK API::MSVCRT.DLL::_amsg_exit FUNC_END::_a
msg_exit_THUNK FUNC_START::FUN_00406c3f FUNC_START::guard_check_icall FUNC_END::guard_
check_icall FUNC_END::FUN_00406c3f FUNC_START::initterm_THUNK API::MSVCRT.DLL::initter
m FUNC_END::initterm_THUNK FUNC_START::__IsNonwritableInCurrentImage FUNC_START::__Val
idateImageBase FUNC_END::__ValidateImageBase FUNC_START::__FindPESection FUNC_END::__F
indPESection FUNC_END::__IsNonwritableInCurrentImage FUNC_START::FUN_00402bfb API::KER
NEL32.DLL::GetVersion API::KERNEL32.DLL::GetModuleHandle…'
```


api_sequence를 계층적 구조로 변환

행위 모델 - Modeling

- 모델 선정

```
--- Sequence Length Distribution ---
count    30859.000000
mean       272.119479
std        497.814967
min          2.000000
50%         33.000000
75%        346.000000
90%        703.200000
95%       1100.000000
99%       2837.000000
max       5763.000000
Name: sequence_length, dtype: float64
```



※계층적 구조로 변환된 api_sequence word 분포

BigBird

PyTorch | Copy page

BigBird is a transformer model built to handle sequence lengths up to 4096 compared to 512 for BERT. Traditional transformers struggle with long inputs because attention gets really expensive as the sequence length grows. BigBird fixes this by using a sparse attention mechanism, which means it doesn't try to look at everything at once. Instead, it mixes in local attention, random attention, and a few global tokens to process the whole input. This combination gives it the best of both worlds. It keeps the computation efficient while still capturing enough of the sequence to understand it well. Because of this, BigBird is great at tasks involving long documents, like question answering, summarization, and genomic applications.

You can find all the original BigBird checkpoints under the Google organization.

평균 272개의 단어가 분포하지만 용량이 큰 파일들을 수집할 경우를 대비하여 최대 입력 길이가 4096 token인 bigbird 모델 선정

행위 모델 - Modeling

- 모델 학습 과정

```
pre_df['target'].value_counts()
```

target	count
1	5257
0	2567

dtype: int64

```
Epoch 5 Average Training Loss: 0.0801
Validation Metrics: Acc: 0.9365, F1: 0.9568, Loss: 0.3147
Saved best model with F1: 0.9568

--- Final Evaluation on Test Set ---
Loaded best model for testing
Loss: 0.3900
Accuracy: 0.9258
F1: 0.9495
Precision: 0.9165
Recall: 0.9850
```

```
[ ] pre_df['target'].value_counts()
```

target	count
0	10573
1	7356

dtype: int64

```
Epoch 5 Average Training Loss: 0.0689
Validation Metrics: Acc: 0.9208, F1: 0.9230, Loss: 0.4378

--- Final Evaluation on Test Set ---
Loaded best model for testing
Loss: 0.2363
Accuracy: 0.9421
F1: 0.9396
Precision: 0.9742
Recall: 0.9073
```

```
pre_df['target'].value_counts()
```

target	count
1	21912
0	10573

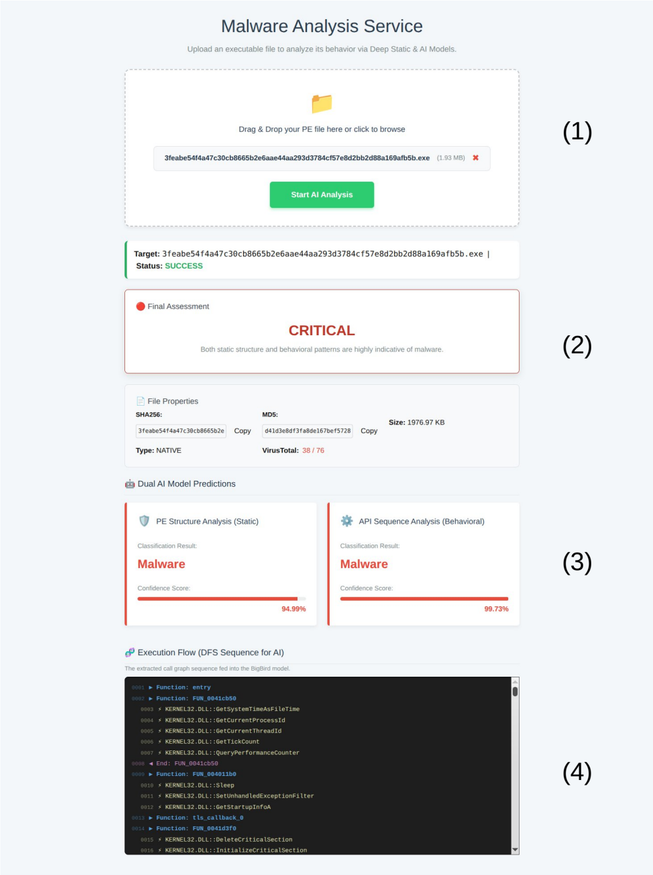
dtype: int64

```
Epoch 10 Average Training Loss: 0.0290
Validation Metrics: Acc: 0.9364, F1: 0.9528, Loss: 0.2900

--- Final Evaluation on Test Set ---
Loading best model from '/content/drive/MyDrive/3m_project
Loss: 0.1455
Accuracy: 0.9624
F1: 0.9731
Precision: 0.9718
Recall: 0.9745
```

학습 데이터수가 증가할수록 평가 값들이 증가하고
최종 모델은 F1 score: 0.9731 모델을 채택함

웹 서비스

- **웹사이트** UI



(1)

(2)

(3)

(4)

(1) 최종 종합 평가

이 섹션은 최종 결론을 제시하는 가장 중요한 부분입니다. 단순히 개별 모델의 예측 결과를 나열하는 것을 넘어, 두 AI 모델의 예측 결과, VirusTotal 탐지율, 위험 API 호출 여부 등 모든 분석 지표를 종합하여 자체적인 로직에 따라 최종 위험도를 Critical에서 Low까지의 등급으로 자동 평가합니다. 이를 통해 사용자는 파일의 위험 수준을 즉각적이고 명확하게 인지할 수 있습니다.

(2) 파일 속성 및 외부 평판

여기서는 분석된 파일의 고유 식별 정보 크기, 파일 타입 등 기본적인 프로필을 제공합니다. 특히, 저희는 파일의 SHA256
해시를 VirusTotal API에 실시간으로 조회하여, 전 세계 수십 개의 상용 안티바이러스 엔진이 이 파일을 어떻게 평가하는지에 대한 객관적인 외부 평판 정보를 함께 제공합니다. 이는 저희 자체 모델의 예측 결과를 교차 검증하고 신뢰도를 높이는 중요한 역할을 합니다.

(3) 하이브리드 AI 모델 예측

각 모델의 예측과 신뢰도 점수를 시각적으로 보여줌으로써, 저희 시스템이 어떤 근거로 최종 평가를 내렸는지 사용자가 이해할 수 있도록 돕습니다.

- **PE Structure Analysis :** 파일의 구조적, 물리적 특징을 분석한 프로파일러 모델의 예측 결과입니다.
- **API Sequence Analysis :** 예측된 실행 흐름과 API 호출 문맥을 분석한 '행동 분석가' 모델의 예측 결과입니다.

(4) 계층적 API_sequence

마지막으로 Big_bird 모델에 판별되기전 시퀀스 형태입니다. 계층적 구조 형태로 어떤 함수 내에서 어떤 API들이 특정 목적을 위해 뭉쳐서 호출되는지를 시퀀스화했습니다.

문제 및 문제 해결

- 동적분석 환경 구축의 제약

초기 목표로 삼았던 동적 분석 환경, 즉 CAPEv2 샌드박스를 구축하는 과정이었습니다.

[문제 상황]
CAPEv2는 악성코드의 실제 행위를 가장 정확하게 포착할 수 있는 강력한 도구이지만, 이를 운영 환경에 구축하는 것은 단순한 프로그램 설치 이상의 복잡성을 요구했습니다. kvm 가상환경이 동작하는데 어려움을 겪었습니다. 또 분석하는데 오랜 시간이 소요되었습니다.

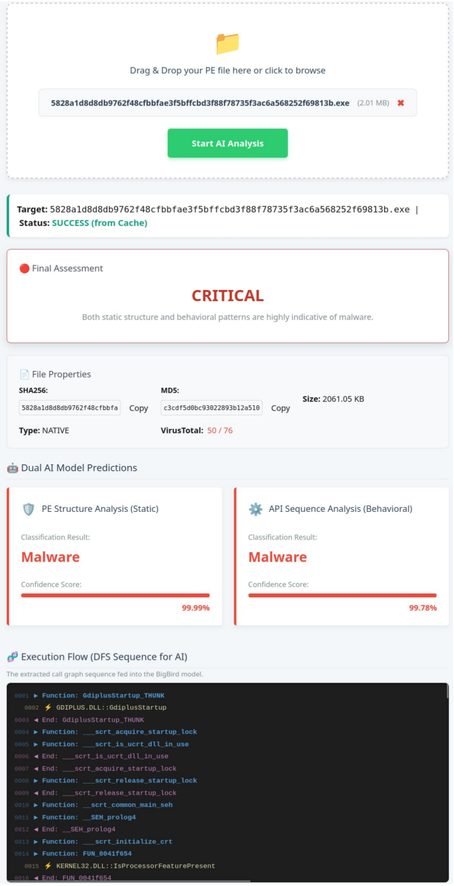
[해결 및 전환]
이 문제를 해결하기 위해 수많은 시도를 했지만, 제한된 프로젝트 기간과 저희의 컴퓨팅 자원 내에서 안정적인 샌드박스 환경을 구축하고 유지하는 것은 비효율적이라는 결론에 도달했습니다.

이 과정에서 완벽한 도구를 고집하기보다 프로젝트의 핵심 목표에 집중하는 것이 더 중요하다는 것을 깨달았습니다. 핵심 목표는 실행 흐름 기반의 행위 분석이었고, Ghidra의 디컴파일 기능을 알게 되었습니다. 그리하여 DFS알고리즘을 결합하여 실제 실행 없이도 동적 행위를 높은 정확도로 예측할 수 있다는 대안을 찾게 되었습니다.

이 경험을 통해 실용적인 해결책을 찾아내는 문제 해결 능력을 기를 수 있었습니다.

향후 계획

- 더 높은 탐지 정확도와 지속 가능한 성장을 위한 향후 계획



모델 고도화: 오탐 감소 및 신뢰도 향상

현재 상황 및 문제점:
현재 저희 AI 모델들은 정상적인 파일의 경우 시스템파일 위주로 학습되었습니다. 그 결과 상용 소프트웨어에 대한 학습이 상대적으로 부족합니다. 왼쪽 그림에서 볼 수 있듯이, LineInst.exe와 같이 복잡한 설치 로직을 가진 정상 프로그램을 악성코드로 오진하는 경우가 발생할 수 있습니다.

향후 계획:
이 오탐 문제를 해결하기 위해, 다양한 종류의 정상 애플리케이션 설치 파일, 게임 클라이언트, 업무용 소프트웨어 등 대규모 정상 파일 데이터셋을 구축할 것입니다. 자동화된 웹 크롤러를 구축하여 공개 소프트웨어 배포 사이트에서 주기적으로 샘플을 수집하고, 이를 AI 모델의 재학습 파이프라인에 자동으로 통합할 계획입니다. 이를 통해 모델은 악성 행위와 정상적인 시스템 설치 행위의 미묘한 차이를 학습하여, 오탐율을 획기적으로 낮추고 전반적인 탐지 신뢰도를 향상시킬 것입니다.